NAAN MUDHALVAN PROJECT REPORT

STUDENT ATTENDANCE TRACKER

A browser-based application for student records and daily attendance management

Submitted by	S. MOHAMMED KALEEM
Register Number	36725U18016
Student / College ID	astvu36736725u18016
Department	B.Sc. Computer Science
Institution	Global College of Arts and Science
Submission Date	30 October 2026

GitHub link
https://github.com/kaleemfreefire9-del/Kaleem.git

DECLARATION

I hereby submit this project report titled **“Student Attendance Tracker”** as documentation of the web application developed using the HTML, CSS and JavaScript source files supplied with this submission. The report describes the implementation, features, data handling, testing approach and current limitations of the application.

Student: S. MOHAMMED KALEEM **Register No.:** 36725U18016

Date: 30 October 2026

ACKNOWLEDGEMENT

I express my sincere thanks to the management, faculty members and department of Global College of Arts and Science for their support and encouragement. I also acknowledge the Naan Mudhalvan initiative for providing a learning opportunity to develop practical technical skills. This project helped me understand how front-end web technologies can be combined to solve a simple academic administration problem.

ABSTRACT

The Student Attendance Tracker is a lightweight, browser-based application designed to maintain student details and record daily attendance. It provides a dashboard showing the total number of students, present students, absent students and an overall attendance percentage. Users can add, edit and delete student profiles; mark attendance as Present, Absent, Late or Leave; review individual attendance history; search and filter records; and identify students whose attendance percentage is below a configurable threshold.

The application is implemented with three client-side files: **index.html** defines the page structure, **style.css** controls layout and visual presentation, and **script.js** implements application logic. Data is stored in the browser's **localStorage** as JSON, so records remain available in the same browser on the same device. No server or external database is included in the supplied code.

KEYWORDS

Student information system, attendance management, HTML, CSS, JavaScript, DOM manipulation, localStorage, CRUD operations, responsive web design.

TABLE OF CONTENTS

Section	Title
1	Introduction
2	Problem Statement and Objectives
3	Scope and Existing / Proposed System
4	Software and Hardware Requirements
5	System Design and Architecture
6	Database / Data Storage Design
7	Module Description
8	Implementation Details
9	Algorithms and Attendance Calculation
10	Testing and Validation
11	How to Run the Project
12	Results, Limitations and Future Enhancements
13	Conclusion
14	References
Appendix A	Complete HTML Source
Appendix B	Complete CSS Source
Appendix C	Complete JavaScript Source

1. INTRODUCTION

Attendance is an important part of academic administration. Maintaining attendance in paper registers can take time, makes searching older records difficult and requires manual calculation of percentages. A simple digital tool can make these tasks faster and more consistent.

The Student Attendance Tracker provides a single web page for managing student profiles and their daily attendance. It is intended as a small academic project that demonstrates form handling, event listeners, DOM updates, data validation, browser storage and responsive user interface design.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem statement

A manual attendance register may become difficult to maintain when student details, daily status and attendance history must be checked repeatedly. Manual percentage calculation can also lead to errors. The project addresses this by bringing student records, attendance marking and summary information into one browser-based interface.

2.2 Objectives

- Provide a form to add new students with a unique student ID, name and department.
- Allow existing student information to be edited and student records to be deleted.
- Record one attendance status per student for a selected date.
- Support Present, Absent, Late and Leave attendance statuses.
- Calculate attendance percentage using Present and Absent records.
- Show daily summary counts and an overall average attendance percentage.
- Search students by ID, name or department and filter by selected-date status.
- Display and remove individual attendance history records.
- Highlight students whose percentage is below a user-defined minimum threshold.
- Save records in browser localStorage so they persist across page reloads on that browser.

3. SCOPE AND SYSTEM OVERVIEW

3.1 Scope

The supplied implementation is a front-end application for a single browser environment. It is suitable for demonstration, learning and small local use. It does not provide user login, multi-user access, cloud synchronisation, server-side storage or automated import/export.

3.2 Existing system

In a paper-based process, student information and attendance are recorded manually. Finding a student's past attendance and calculating percentages may require checking multiple pages and doing arithmetic by hand.

3.3 Proposed system

The proposed system displays a dashboard and student table. Users can maintain student profiles, mark attendance for a selected date, filter the table and inspect history. JavaScript updates the page immediately and stores changes in localStorage.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Category	Requirement
Frontend	HTML5, CSS3 and vanilla JavaScript
Storage	Browser localStorage (JSON)
Runtime	Modern web browser such as Chrome, Edge or Firefox
Editor (optional)	Visual Studio Code or any text editor
Hardware	Computer or smartphone capable of running a modern browser
Internet	Not required after the project files are available; no external libraries are used

5. SYSTEM DESIGN AND ARCHITECTURE

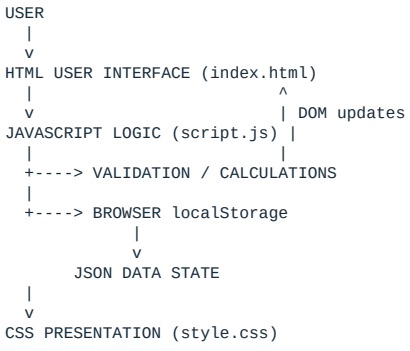
The application follows a simple client-side architecture. The browser loads the HTML page, applies the CSS rules and executes JavaScript. The JavaScript layer reads and writes the attendance data in localStorage and renders the current state into the HTML table and dashboard.

Layer	File / Component	Responsibility
Presentation	index.html	Defines header, summary cards, student form, attendance controls, tables and history panel.
Styling	style.css	Defines colours, spacing, cards, buttons, tables and responsive layouts.
Application logic	script.js	Validates input, manages student and attendance data, calculates percentages and renders the UI.
Persistence	localStorage	Stores the JSON object under the key attendanceTrackerData.

5.1 High-level workflow

- Open index.html in a browser.
- JavaScript loads saved JSON from localStorage, or starts with empty student and attendance arrays.
- The user adds student details or selects an existing student.
- The user selects a date and marks a status.
- The application validates the operation, updates the state and saves it to localStorage.
- The render function refreshes dashboard values, filtered student rows and an open history panel.

5.2 Textual architecture diagram



6. DATA STORAGE DESIGN

The application stores a single JSON object in localStorage using the key **attendanceTrackerData**. The object contains two arrays: students and attendance.

6.1 Student record

```
{
  "studentId": "36725U18016",
  "name": "Example Student",
  "department": "B.Sc. CS"
}
```

6.2 Attendance record

```
{
  "studentId": "36725U18016",
  "date": "2026-10-30",
  "status": "Present"
}
```

6.3 Storage characteristics

- Student IDs are used to connect attendance records to student records.
- The application prevents duplicate student IDs without regard to letter case.
- When a student ID is edited, matching attendance records are updated to the new ID.
- When a student is deleted, the application also deletes that student's attendance records.
- Data is local to the browser profile and device; clearing browser site data can remove it.

7. MODULE DESCRIPTION

7.1 Dashboard

Displays total registered students, present count for the selected date, absent count for the selected date and average student attendance percentage.

7.2 Add Student

Accepts student ID, name and department. Empty fields and duplicate IDs are rejected.

7.3 Edit Student

Loads an existing record into the form and updates its details. Attendance references are kept consistent if the ID changes.

7.4 Delete Student

Asks for confirmation, then removes the student and all attendance records linked to that ID.

7.5 Daily Attendance

Allows a date to be selected and one of four statuses—Present, Absent, Late or Leave—to be recorded for each student.

7.6 Search and Filter

Searches across student ID, name and department, and filters students by their status on the selected date.

7.7 Attendance Percentage

Counts only Present and Absent records. Late and Leave do not enter the percentage denominator in the current implementation.

7.8 Low Attendance Warning

Highlights rows when a student has at least one Present/Absent record and the calculated percentage is lower than the configured threshold.

7.9 Attendance History

Shows records for one student in descending date order and allows an individual record to be removed.

8. IMPLEMENTATION DETAILS

8.1 HTML structure

The HTML document includes a page header, four summary cards, a student form, attendance controls, a student table and a hidden attendance-history panel. Semantic sections and labels are used to organise the page. The script is loaded at the end of the body so the page elements exist before the JavaScript attaches event listeners.

8.2 CSS design

The stylesheet uses a dark blue header, light page background, white cards and panels, and coloured status buttons. The summary area uses a four-column grid on wider screens, two columns on medium screens and one column on narrow screens. At widths below 760 pixels, forms and control bars become vertical to improve mobile usability.

8.3 JavaScript logic

The script uses a small state object, helper functions and event delegation. The render() function is the central UI refresh operation. Instead of attaching separate click handlers to every generated button, click events are handled on the student table and inspected using data-action attributes.

8.4 Input safety and validation

The code trims user-entered values, checks required fields and rejects duplicate IDs. Dynamic student values inserted into table markup are passed through escapeHTML() to prevent HTML special characters from being interpreted as markup. Attendance marking checks that a date is selected and prevents a second record for the same student and date.

9. ALGORITHMS AND ATTENDANCE CALCULATION

9.1 Attendance percentage

For a given student, the application selects attendance records whose status is Present or Absent. It calculates:

Attendance % = (Number of Present records / (Present records + Absent records)) × 100

The result is rounded to the nearest whole percentage. If there are no Present or Absent records, the function returns 0%. Late and Leave are excluded from this calculation by the supplied code.

9.2 Overall attendance

The dashboard calculates the arithmetic mean of the individual student percentages. If there are no students, the dashboard displays 0%. This is a student-wise average, not a weighted total across all attendance sessions.

9.3 Duplicate attendance prevention

- Read the selected date and student ID.

- Search the attendance array for a record with the same student ID and date.
- If a record exists, show an error and do not add another record.
- Otherwise append the new record, save the state and refresh the display.

9.4 Low attendance condition

A row is highlighted when the student has at least one Present or Absent record and their calculated percentage is less than the threshold entered by the user. The default threshold is 75%.

10. TESTING AND VALIDATION

The following test cases are recommended for manual verification in a browser. Expected outcomes are based on the supplied JavaScript implementation.

Test ID	Test input / action	Expected result
TC-01	Add student with all fields	Student appears in the table and total count increases.
TC-02	Submit empty student form	Browser required-field validation prevents submission.
TC-03	Add duplicate student ID	Error message indicates that the ID already exists.
TC-04	Edit a student record	Updated name, ID or department appears; attendance links follow an ID change.
TC-05	Delete a student	Confirmation is shown; student and linked attendance are removed.
TC-06	Mark Present for a selected date	Status is shown and present count increases.
TC-07	Mark a second status for same student/date	Duplicate-record error is displayed; existing status remains.
TC-08	Mark Absent, Late and Leave	The selected status is displayed; daily absent count increases only for Absent.
TC-09	Search by name, ID or department	Only matching student rows are displayed.
TC-10	Filter by status	Rows match the selected-date status filter.
TC-11	Open student history	Records appear newest date first.
TC-12	Remove history record	Selected attendance record is removed and values recalculate.
TC-13	Change threshold	Rows below the new threshold are highlighted.
TC-14	Reload the browser page	Previously saved data is restored from localStorage.

Testing note: These are defined test scenarios, not a claim that automated tests were executed. The supplied ZIP contains application source files but no test automation or test-result log.

11. HOW TO RUN THE PROJECT

- Extract Kaleem_Attendance_Tracker_Project.zip to a folder.
- Confirm that index.html, style.css and script.js are in the same folder.

- Open index.html using a modern web browser. No installation or build step is required.
- Add student records using the Add Student form.
- Choose the attendance date and use the Present, Absent, Late or Leave buttons.
- Use Search, the status filter and the History action to find and review records.
- Keep using the same browser profile to retain the localStorage data.

11.1 Project file structure

```
Kaleem_Attendance_Tracker_Project/
|-- index.html
|-- style.css
|-- script.js
```

12. RESULTS, LIMITATIONS AND FUTURE ENHANCEMENTS

12.1 Expected result

When opened in a browser, the project presents a responsive attendance dashboard. Student details can be maintained, daily attendance can be marked and reviewed, and summary percentages are recalculated after changes. The interface provides search, status filtering, history review and low-attendance highlighting.

12.2 Current limitations

- Records are stored only in the current browser's localStorage; there is no central database or automatic backup.
- There is no login, role-based access control or audit trail.
- The system does not synchronise records between different devices or users.
- The application prevents duplicate student/date records, but it does not support editing an existing attendance record directly; the record must be removed from History and marked again.
- Late and Leave are recorded and displayed but are excluded from the attendance percentage calculation.
- The date is generated using toISOString(), which uses UTC; around local midnight this may differ from the user's local calendar date.
- No export to CSV/PDF, bulk import, timetable or monthly report feature is included in the supplied code.

12.3 Future enhancements

- Connect a secure backend and database such as MySQL or PostgreSQL.
- Add administrator and faculty login with role-based permissions.
- Add backup, restore and CSV/PDF export facilities.
- Provide monthly, semester-wise and subject-wise attendance reports.
- Allow authorised editing of attendance with an audit log.
- Define an institutional policy for how Late and Leave should affect attendance percentage.
- Use local date handling and add automated unit and browser tests.
- Add accessibility improvements, pagination and bulk student import.

13. CONCLUSION

The Student Attendance Tracker demonstrates how HTML, CSS and JavaScript can be used to build a practical browser-based attendance management tool. It combines student CRUD operations, daily attendance marking, filtering, history and percentage calculation in a responsive interface. Browser localStorage provides simple persistence without requiring a server. The project meets its learning objective as a front-end prototype, while a production deployment would require secure authentication, central storage, backups and stronger operational controls.

14. REFERENCES

- MDN Web Docs – HTML: https://developer.mozilla.org/en-US/docs/Web/HTML
- MDN Web Docs – CSS: https://developer.mozilla.org/en-US/docs/Web/CSS
- MDN Web Docs – JavaScript: https://developer.mozilla.org/en-US/docs/Web/JavaScript
- MDN Web Docs – Window.localStorage: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
- Project source files supplied in Kaleem_Attendance_Tracker_Project.zip (index.html, style.css and script.js).

GitHub repository: https://github.com/kaleemfreefire9-del/Kaleem.git

APPENDIX A – COMPLETE HTML SOURCE CODE

The following listing is reproduced from the supplied project ZIP. It is included so the report documents the actual implementation.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Attendance Tracker</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Student Attendance Tracker</h1>
    <p>Global College of Arts and Science | B.Sc. (Computer Science)</p>
  </header>
  <main>
    <section class="summary" aria-label="Attendance summary">
      <article class="card"><span>Total Students</span><strong id="total">0</strong></article>
      <article class="card"><span>Present Today</span><strong id="present">0</strong></article>
      <article class="card"><span>Absent Today</span><strong id="absent">0</strong></article>
      <article class="card"><span>Overall Attendance</span><strong id="overall">0%</strong></article>
    </section>
    <section class="panel">
      <h2 id="formTitle">Add Student</h2>
      <form id="studentForm">
        <input type="hidden" id="editOriginalId">
        <input id="studentId" placeholder="Student ID (e.g., 36725U18016)" required>
        <input id="studentName" placeholder="Student Name" required>
        <input id="department" placeholder="Department (e.g., B.Sc. CS)" required>
        <button type="submit" id="submitBtn">Add Student</button>
        <button type="button" id="cancelEditBtn" class="btn-secondary hidden">Cancel</button>
      </form>
    </section>
    <section class="panel">
      <h2>Mark Daily Attendance &amp;amp; Records</h2>
      <div class="controls-bar">
        <div class="control-group"><label for="attendanceDate">Date:</label><input id="attendanceDate"
          type="date" required></div>
        <div class="control-group"><label for="threshold">Min Threshold (%):</label><input id="threshold"
          type="number" min="0" max="100" value="75" class="small-input"></div>
        <div class="control-group grow"><input id="search" placeholder="Search by name, ID or
          department..."></div>
        <div class="control-group"><select id="statusFilter"><option value="All">All
          Statuses</option><option value="Present">Present</option><option
          value="Absent">Absent</option><option value="Late">Late</option><option
          value="Leave">Leave</option><option value="Not marked">Not Marked</option></select></div>
      </div>
      <p id="message" role="status"></p>
      <div class="table-wrap"><table><thead><tr><th>Student ID</th><th>Student
        Name</th><th>Department</th><th>Percentage</th><th>Status (Selected Date)</th><th>Mark
        Attendance</th><th>Manage</th></tr></thead><tbody id="studentRows"></tbody></table></div>
    </section>
    <section id="historyPanel" class="panel hidden">
      <div class="history-header"><h2 id="historyTitle">Attendance History</h2><button type="button"
        id="closeHistoryBtn" class="btn-secondary">Close</button></div>
      <div
        class="table-wrap"><table><thead><tr><th>Date</th><th>Status</th><th>Action</th></tr></thead><tbo
        dy id="historyRows"></tbody></table></div>
    </section>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

APPENDIX B – COMPLETE CSS SOURCE CODE

The following listing is reproduced from the supplied project ZIP. It is included so the report documents the actual implementation.

```
* { box-sizing: border-box; }
body { margin: 0; font-family: Arial, sans-serif; background: #f3f6fb; color: #263548; }
header { background: #173656; color: white; padding: 24px 5%; }
header h1 { margin: 0 0 8px; }
header p { margin: 0; color: #dce9f8; }
main { max-width: 1200px; margin: 24px auto; padding: 18px; }
.summary { display: grid; grid-template-columns: repeat(4, 1fr); gap: 14px; margin-bottom: 20px; }
.card, .panel { background: white; border: 1px solid #d8e1ed; border-radius: 12px; padding: 18px; }
.card span { display: block; color: #65758b; font-size: 13px; text-transform: uppercase; }
.card strong { display: block; font-size: 28px; margin-top: 10px; color: #1c4e80; }
.panel { margin-bottom: 18px; }
.panel h2 { margin-top: 0; font-size: 18px; color: #173656; }
form, .controls-bar { display: flex; flex-wrap: wrap; gap: 12px; align-items: center; }
.control-group { display: flex; align-items: center; gap: 6px; }
.control-group.grow { flex: 1; }
.control-group.grow input { width: 100%; }
input, select, button { padding: 10px 12px; border: 1px solid #cbd6e4; border-radius: 7px; font-size:
  14px; }
input { min-width: 150px; }
input.small-input { min-width: 70px; width: 75px; }
button { background: #2f75b5; color: white; border: 0; cursor: pointer; transition: background 0.2s; }
button:hover { background: #245d91; }
.btn-present { background: #217346; } .btn-present:hover { background: #195935; }
.btn-absent { background: #b42318; } .btn-absent:hover { background: #8c1b12; }
.btn-late { background: #d97706; } .btn-late:hover { background: #b45309; }
.btn-leave { background: #64748b; } .btn-leave:hover { background: #475569; }
.btn-secondary { background: #e2e8f0; color: #1e293b; } .btn-secondary:hover { background: #cbd5e1; }
.btn-danger { background: #fee2e2; color: #b42318; } .btn-danger:hover { background: #fecaca; }
.btn-sm { padding: 6px 10px; font-size: 12px; margin-right: 4px; }
.table-wrap { overflow-x: auto; margin-top: 14px; }
table { width: 100%; border-collapse: collapse; background: white; }
th, td { padding: 12px; text-align: left; border-bottom: 1px solid #e4eaf2; font-size: 14px; }
th { background: #eaf1f9; color: #173656; }
tr.low-row { background-color: #fffbfa; }
.status-present { color: #217346; font-weight: bold; }
.status-absent, .low { color: #b42318; font-weight: bold; }
.status-late { color: #d97706; font-weight: bold; }
.status-leave { color: #64748b; font-weight: bold; }
#message { min-height: 20px; margin: 10px 0 0; font-weight: bold; color: #217346; }
#message.error { color: #b42318; }
.hidden { display: none; }
.history-header { display: flex; justify-content: space-between; align-items: center; }
@media (max-width: 760px) { .summary { grid-template-columns: repeat(2, 1fr); } form, .controls-bar {
  flex-direction: column; align-items: stretch; } input, select, button { width: 100%; } }
@media (max-width: 420px) { .summary { grid-template-columns: 1fr; } }
```

APPENDIX C – COMPLETE JAVASCRIPT SOURCE CODE

The following listing is reproduced from the supplied project ZIP. It is included so the report documents the actual implementation.

```
const KEY = "attendanceTrackerData";
const state = JSON.parse(localStorage.getItem(KEY) || '{"students": [], "attendance": []}');
const $ = id => document.getElementById(id);
const today = new Date().toISOString().slice(0, 10);
$("attendanceDate").value = today;
let activeHistoryStudentId = null;
function save() { localStorage.setItem(KEY, JSON.stringify(state)); }
function showMessage(text, isError = false) {
  const msgEl = $("message"); msgEl.textContent = text; msgEl.className = isError ? "error" : "";
}
function escapeHTML(value) {
  return String(value).replace(/[&<>"']/g, ch => ({
    "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;"
  }[ch]));
}
function saveStudent(originalId, studentId, name, department) {
  studentId = studentId.trim(); name = name.trim(); department = department.trim();
  if (!studentId || !name || !department) throw new Error("All student fields are required.");
  if (originalId) {
    if (originalId !== studentId && state.students.some(s => s.studentId.toLowerCase() ===
      studentId.toLowerCase())) throw new Error("New Student ID already exists.");
    const student = state.students.find(s => s.studentId === originalId);
    if (student) {
      student.studentId = studentId; student.name = name; student.department = department;
      state.attendance.forEach(a => { if (a.studentId === originalId) a.studentId = studentId; });
    }
  } else {
    if (state.students.some(s => s.studentId.toLowerCase() === studentId.toLowerCase())) throw new
      Error("Student ID already exists.");
    state.students.push({ studentId, name, department });
  }
  save();
}
function deleteStudent(studentId) {
  if (!confirm(`Delete student ${studentId} and all their attendance records?`)) return;
  state.students = state.students.filter(s => s.studentId !== studentId);
  state.attendance = state.attendance.filter(a => a.studentId !== studentId);
  if (activeHistoryStudentId === studentId) { $("historyPanel").classList.add("hidden");
    activeHistoryStudentId = null; }
  save(); showMessage("Student deleted."); render();
}
function startEditStudent(studentId) {
  const student = state.students.find(s => s.studentId === studentId); if (!student) return;
  $("editOriginalId").value = student.studentId; $("studentId").value = student.studentId;
  $("studentName").value = student.name; $("department").value = student.department;
  $("formTitle").textContent = "Edit Student"; $("submitBtn").textContent = "Update Student";
  $("cancelEditBtn").classList.remove("hidden"); $("studentId").focus();
}
function resetForm() {
  $("studentForm").reset(); $("editOriginalId").value = "";
  $("formTitle").textContent = "Add Student"; $("submitBtn").textContent = "Add Student";
  $("cancelEditBtn").classList.add("hidden");
}
function markAttendance(studentId, date, status) {
  if (!date) throw new Error("Please select a valid date.");
  const existing = state.attendance.find(a => a.studentId === studentId && a.date === date);
  if (existing) throw new Error(`Attendance already recorded as '${existing.status}' for ${date}. Delete
    it from History to re-mark.`);
  state.attendance.push({ studentId, date, status }); save();
}
function percentage(studentId) {
  const rows = state.attendance.filter(a => a.studentId === studentId && ["Present",
    "Absent"].includes(a.status));
  if (!rows.length) return 0;
  return Math.round(rows.filter(a => a.status === "Present").length / rows.length * 100);
}
function getStatusClass(status) {
  if (status === "Present") return "status-present";
  if (status === "Absent") return "status-absent";
  if (status === "Late") return "status-late";
  if (status === "Leave") return "status-leave";
  return "";
}
function renderHistory(studentId) {
  const student = state.students.find(s => s.studentId === studentId); if (!student) return;
  activeHistoryStudentId = studentId;
  $("historyTitle").textContent = `Attendance History: ${student.name} (${student.studentId})`;
  const records = state.attendance.filter(a => a.studentId === studentId).sort((a,b) =>
    b.date.localeCompare(a.date));
  $("historyRows").innerHTML = !records.length ? `<tr><td colspan="3">No attendance records
    found.</td></tr>` :
    records.map(r => `<tr><td>${escapeHTML(r.date)}</td><td
      class="${getStatusClass(r.status)}">${escapeHTML(r.status)}</td><td><button class="btn-danger
      btn-sm" data-del-date="${escapeHTML(r.date)}" data-del-id="${escapeHTML(r.studentId)}">Remove
      Record</button></td></tr>`).join("");
  $("historyPanel").classList.remove("hidden");
}
function render() {
  const date = $("attendanceDate").value, q = $("search").value.toLowerCase().trim();
```

```
  const statusFilter = $("statusFilter").value, threshold = Number($("threshold").value) || 75;
  const filteredStudents = state.students.filter(s => {
    const matchesSearch = [s.studentId, s.name, s.department].join(" ").toLowerCase().includes(q);
    const saved = state.attendance.find(a => a.studentId === s.studentId && a.date === date);
    const currentStatus = saved ? saved.status : "Not marked";
    return matchesSearch && (statusFilter === "All" || currentStatus === statusFilter);
  });
  $("studentRows").innerHTML = !filteredStudents.length ? `<tr><td colspan="7">No matching student
    records found.</td></tr>` :
    filteredStudents.map(s => {
      const saved = state.attendance.find(a => a.studentId === s.studentId && a.date === date);
      const status = saved ? saved.status : "Not marked", pct = percentage(s.studentId);
      const hasRecords = state.attendance.some(a => a.studentId === s.studentId && ["Present",
        "Absent"].includes(a.status));
      const isLow = hasRecords && pct < threshold;
      return `<tr class="${isLow ? 'low-row' : ''}">
<td>${escapeHTML(s.studentId)}</td><td>${escapeHTML(s.name)}</td><td>${escapeHTML(s.department)}</td>
        <td class="${isLow ? 'low' : 'status-present'}">${pct}% ${isLow ? '(Low)' : ''}</td>
        <td class="${getStatusClass(status)}">${escapeHTML(status)}</td>
        <td><button class="btn-present btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}"
          data-status="Present">Present</button>
        <button class="btn-absent btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}"
          data-status="Absent">Absent</button>
        <button class="btn-late btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}"
          data-status="Late">Late</button>
        <button class="btn-leave btn-sm" data-action="mark" data-id="${escapeHTML(s.studentId)}"
          data-status="Leave">Leave</button></td>
        <td><button class="btn-secondary btn-sm" data-action="history"
          data-id="${escapeHTML(s.studentId)}">History</button>
        <button class="btn-secondary btn-sm" data-action="edit"
          data-id="${escapeHTML(s.studentId)}">Edit</button>
        <button class="btn-danger btn-sm" data-action="delete"
          data-id="${escapeHTML(s.studentId)}">Delete</button></td></tr>`;
    }).join("");
  const todays = state.attendance.filter(a => a.date === date);
  $("total").textContent = state.students.length;
  $("present").textContent = todays.filter(a => a.status === "Present").length;
  $("absent").textContent = todays.filter(a => a.status === "Absent").length;
  const percentages = state.students.map(s => percentage(s.studentId));
  $("overall").textContent = percentages.length ? Math.round(percentages.reduce((a,b) =>
    a+b,0)/percentages.length)+"%" : "0%";
  if (activeHistoryStudentId) renderHistory(activeHistoryStudentId);
}
$("studentForm").addEventListener("submit", event => {
  event.preventDefault();
  try {
    const origId = $("editOriginalId").value;
    saveStudent(origId, $("studentId").value, $("studentName").value, $("department").value);
    showMessage(origId ? "Student updated successfully." : "Student added successfully."); resetForm();
      render();
  } catch (error) { showMessage(error.message, true); }
});
$("cancelEditBtn").addEventListener("click", resetForm);
$("studentRows").addEventListener("click", event => {
  const button = event.target.closest("button[data-action]"); if (!button) return;
  const { action, id, status } = button.dataset;
  try {
    if (action === "mark") { markAttendance(id, $("attendanceDate").value, status); showMessage(`Marked
      ${status} for ${id}.`); render(); }
    else if (action === "edit") startEditStudent(id);
    else if (action === "delete") deleteStudent(id);
    else if (action === "history") renderHistory(id);
  } catch (error) { showMessage(error.message, true); }
});
$("historyRows").addEventListener("click", event => {
  const button = event.target.closest("button[data-del-date]"); if (!button) return;
  const { delId, delDate } = button.dataset;
  state.attendance = state.attendance.filter(a => !(a.studentId === delId && a.date === delDate));
  save(); showMessage(`Removed attendance record for ${delDate}.`); render();
});
$("closeHistoryBtn").addEventListener("click", () => { $("historyPanel").classList.add("hidden");
  activeHistoryStudentId = null; });
$("search").addEventListener("input", render); $("statusFilter").addEventListener("change", render);
$("threshold").addEventListener("input", render); $("attendanceDate").addEventListener("change", render);
render();
```